

SEMV Warehouse — Technical Guide

Implementation reference • 9 September 2026

This document describes the code in this project. It covers the interactive warehouse plan, the three demonstration apps, animation systems, responsive interface, browser navigation, assets, build tools and verification. It is intended for developers maintaining the demo and for reviewers explaining how it works.

1. What the application is

SEMV Warehouse is a static, browser-based demonstration of Goods Received (GR), PutAway and Pallet Interlocking. Visitors explore an animated warehouse, open a station, try its simulated workflow and follow a guided three-stage journey. A separate Project Detail view explains the project and contains a manual carton-matching challenge.

The warehouse is an isometric SVG scene. Objects have model coordinates and height, then are projected onto a two-dimensional display. This gives a three-dimensional appearance without WebGL or a general-purpose 3D engine. Dragging pans the plan; it does not orbit a perspective camera.

The application uses sample data and scripted activities. It has no live warehouse connection, SAP integration, production barcode scanner, database, login service or backend API. Completing a demo changes the visitor's local session only. The ambient warehouse logistics and the guided demo sequence are separate simulations.

2. Technology stack

Technology	Use in this project
HTML5	Five application views, semantic controls, terminals, dialogs and project content
CSS	Themes, design tokens, Grid and Flexbox layouts, container queries, media queries, transitions and keyframe animation
JavaScript	Plain browser JavaScript for routing, state, workflows, geometry, interaction and animation
SVG	Warehouse surfaces, operators, vehicles, shadows, pallets, wrapper film and transition illustrations
Browser History API	Fragment routes, Back/Forward handling and page restoration
Pointer Events	Mouse, touch and pen dragging, pinch gestures and pointer capture
requestAnimationFrame	Scheduling movement, operator poses and dynamic geometry updates
Web Animations API	Reading CSS animation time with <code>getAnimations()</code> and coordinating <code>play/pause/currentTime</code> for truck holds

SVG animation APIs	Pausing and resuming SVG animation through <code>pauseAnimations()</code> , <code>unpauseAnimations()</code> and <code>setCurrentTime()</code>
ResizeObserver	Measuring the top bar and fitting the home caption within the available canvas
matchMedia	System theme selection and reduced-motion preference
Page Visibility API	Pausing animation and challenge timing while the page is hidden
Node.js and npm	Development scripts, building the portable preview and running checks
puppeteer-core 25.10.0	Automated browser interaction and screenshots; a development dependency in practice
Microsoft Edge	Browser used by the repository's automated checks; configurable through <code>BROWSER_PATH</code>
Python standard library	Alternative single-file preview builder in <code>build-preview.py</code>
PNG	Locally bundled Schneider Electric and RMIT SSET artwork

The browser does not load React, Vue, Angular, Three.js, GSAP, Bootstrap, a remote font service or Puppeteer. No application framework, transpiler or runtime package installation is needed to open the app. Browser scripts are ordinary script files; Node build and test scripts use ECMAScript modules with the `.mjs` extension.

3. Application structure and startup

The source entry point is `index.html`. It loads CSS and JavaScript in a deliberate order. Each browser module wraps its implementation in an immediately invoked function and exposes a small object on window. This keeps implementation variables private while allowing the modules to collaborate without a bundler.

On `DOMContentLoaded`, `js/main.js` sets Vietnamese as the initial language, selects a theme from the system preference, mounts the warehouse, prepares the project challenge, subscribes to completion updates and initializes routing. It also starts the header and caption measurements. Station demos are created when first visited and their instances are reused during navigation.

```
index.html
├─ main.js: initialization and application coordination
├─ router.js: active view, URL and transitions
├─ state.js: completed station IDs
├─ i18n.js: English / Vietnamese content
├─ demos.js + handoff.js: guided workflow
├─ challenge.js: manual matching exercise
├─ twin.js: warehouse scene and interaction
│   ├─ warehouse-environment.js: floor, shell and details
│   ├─ camera.js: pan and zoom
│   ├─ vehicles.js: rotated vehicle and cargo meshes
│   ├─ logistics.js: movement and cargo ownership
│   ├─ logistics-renderer.js: scene updates and wrapping
│   └─ operators.js: articulated people
```

4. File reference

File	Responsibility
index.html	Source markup, view containers, header, controls and ordered script/style loading
preview-single-file.html	Generated portable app containing inline scripts, styles and logo images
privacy.html / terms.html	Separate informational pages; their placeholder policy copy needs review before publication
js/main.js	Initialization, theme/language controls, demo mounting, progress, guided journey and layout measurement
js/router.js	View switching, browser history, focus management and forklift transition cleanup
js/state.js	WarehouseState completion store and subscriptions
js/i18n.js	Translation dictionaries and DOM translation helpers
js/twin.js	Warehouse geometry, scene assembly, activity loop, station hit areas, operator labels and speech
js/camera.js	SVG viewBox pan/zoom, pointer handling and map controls
js/vehicles.js	Truck trajectories, rotating mesh geometry, trailer doors and dock holds
js/logistics.js	Forklift task programs, cargo transfers, storage occupancy and pedestrian yielding
js/logistics-renderer.js	Rendering model state, dynamic depth, pallet meshes and wrapper animation
js/operators.js	Jointed body geometry, gait, turning, work poses and greetings
js/warehouse-environment.js	Foundation, markings, area outlines, rack stock, shell, services and shadows
js/demos.js	GoodsReceived, PutAway and Interlocking demo factories and sample records
js/challenge.js	Timed manual matching challenge and discrepancy review
js/handoff.js	Animated GR-to-PutAway and PutAway-to-Interlocking handoffs
js/transition-forklift.js	Detailed forklift artwork used when changing views
js/forklift.js	Older forklift helper and shared reduced-motion helper; not the current detailed transition artwork
css/tokens.css	Theme colours, fonts, spacing, sizes and motion constants
css/base.css	Reset, typography, focus treatment and base interactive sizing
css/layout.css	Top bar, partner-logo layout and shared page structures
css/views.css	Fixed application views, station layouts and navigation transition

css/twin.css	Warehouse materials, actors, map controls and SVG component styling
css/stage.css	Caption and stage presentation
css/warehouse-home.css	Current home styling and responsive refinements
css/animations.css	CSS keyframes, dock clocks and motion preference rules
css/components.css	Buttons, terminals, fields, status indicators and reusable UI
css/sections.css / css/project.css	Project and informational page presentation
css/handoff.css	Guided handoff dialog and scene presentation
scripts/build-preview.mjs	Node single-file builder
build-preview.py	Python single-file builder
scripts/check-*.mjs	Targeted automated verification scripts
docs/technical-guide.md	Editable source for this implementation guide
scripts/build-tech-docs.mjs	Produces the Word Open XML package and formatted HTML guide with Node's standard library
scripts/build-tech-pdf.mjs	Exports the HTML guide to PDF with the installed browser
artifacts/	Local screenshots, diagnostic captures and test reports

5. Routing and browser Back

Five route IDs correspond to five section elements: home, gr, putaway, interlock and more. For example, #gr selects #view-gr. Only the selected view has is-active and is visible; body[data-view] also reflects the current view.

Elements marked data-goto delegate navigation to Router.go(). Navigation records a fragment URL through history.pushState(). Initialization and normalization use replaceState(). Fragment URLs work for both file:// previews and static HTTP hosting, without server-side route rewriting. Unknown route fragments resolve to home. Reloading a route opens its matching view; it does not persist the demo's in-memory progress.

Back and Forward are handled through popstate and hashchange. History restoration cancels outstanding transition timers and any guided handoff before applying the destination. This prevents a delayed animation callback from returning the visitor to a screen they already left. pagehide clears transient animations, while a persisted pageshow resynchronizes the restored view after browser-cache restoration.

An ordinary animated route change lasts 1,700 ms. The screen changes behind the curtain at 780 ms. Reduced motion and explicit instant navigation bypass that animation. Navigation updates the selected top-bar link, moves focus to the view heading where available and notifies subscribers so the correct scene and demo resume.

6. State and event flow

WarehouseState stores an array of completed station IDs. Its public operations are `complete()`, `reset()`, `isDone()`, `count()`, `allDone()` and `subscribe()`. Completing an already completed station is ignored, which prevents duplicate progress updates. Subscribers receive the completed app, count and all-complete status.

Each demo maintains its own closure-based state and exposes `render()` and `reset()`. The main module reuses these instances when navigating Back and Forward within the same document. Completion state, selected language and other UI values are not written to `localStorage`, `sessionStorage`, cookies or a database. A fresh page load starts a fresh demo session. Browser cache restoration may retain the existing document and its memory, but is not durable storage.

Click handlers mostly use event delegation with data attributes such as `data-act`, `data-goto`, `data-bin` and `data-node-app`. This allows a terminal's contents to be rerendered without installing another click handler for each generated button. Translation changes dispatch a `languagechange` event to refresh visible content and accessible labels.

7. The three demo workflows

Stage	Simulated behaviour	Main implementation
Goods Received	Opens a receiving record, selects a docket, scans sample boxes, demonstrates duplicate handling, records returnable empties and captures role confirmations	<code>GoodsReceived()</code> in <code>js/demos.js</code>
PutAway	Scans a sample pallet, displays rack bins and a suggested destination, permits selection and confirms placement	<code>PutAway()</code> in <code>js/demos.js</code>
Pallet Interlocking	Selects a pallet, checks handling-unit and EAN information against a sample packing list, demonstrates a mismatch and confirms the checked pallet	<code>Interlocking()</code> in <code>js/demos.js</code>

Barcode bars and matrix graphics are visual illustrations. The demo advances through on-screen actions; it does not use a camera, decode a real barcode or send a warehouse transaction. The suggested PutAway bin is predefined demo data rather than an operational slotting or optimization algorithm.

The Start Journey control opens the first unfinished stage. When all stages are complete, starting again resets the demonstration. GR completion triggers an operator carrying a carton to a rack, then PutAway opens. PutAway completion triggers a forklift retrieving a pallet and transferring it to the wrapper, then Pallet Interlocking opens. Completing all three returns to the warehouse using the detailed forklift transition.

The GR handoff lasts approximately 7 seconds; the PutAway handoff approximately 13 seconds.

`WarehouseHandoff.play()` returns a Promise that reports whether to continue. The dialog makes the background inert, focuses the skip control, handles Escape and restores the previous focus on cleanup. Navigation

cancellation resolves without advancing the workflow. Hidden pages pause the handoff clock; reduced motion skips the animated sequence.

8. Warehouse geometry and depth

The main scene uses a 112 × 66 plan-unit footprint. Its projection in js/twin.js is:

```
screenX = (x - y) × 0.866 × 6.1 + 430  
screenY = (x + y) × 0.5 × 6.1 - z × 6.1 + 74
```

x and y describe a location on the plan, and z describes height. The `box()` helper projects the visible top, right and left faces of a rectangular object into SVG polygons. Material classes give those faces different colours to imply lighting. Mesh rotation happens in plan coordinates before projection, so a turning truck changes direction while its vertical parts remain upright.

SVG paints later elements over earlier elements. The code combines geometric depth sorting with explicitly ordered equipment and actor layers. Some aisle operators are intentionally drawn behind racks or container walls. A visual overlap can therefore be intentional occlusion rather than a collision. When moving an object, review both its model position and its placement in the SVG paint order.

The warehouse floor surface is at $z = 0.7$. Rack footplates and stack supports meet that surface. Floor markings are drawn before equipment contact shadows, so markings cannot erase the shadows that make equipment look grounded. The wrapping platform has a solid cylindrical side and a top at $z = 1.75$. Its pallet base meets that top. The handoff illustration uses its own projection and floor height, so its coordinates must not be copied directly from the main warehouse.

9. Camera and operator interaction

WarehouseCamera changes the SVG `viewBox`. The base `viewBox` is 58 -45 988 690, with an initial zoom of 1.04. Zoom is bounded from 1 to 3 and panning is clamped to retain part of the building in view. Screen coordinates are converted to SVG coordinates using the inverse screen transformation matrix, keeping zoom centered on the interaction point.

The map supports pointer dragging, touch pinch, wheel zoom and three visible controls. A movement threshold separates a drag from a click, and pointer capture keeps a gesture stable outside the original target. When the map has keyboard focus, arrow keys pan, + or = zooms in, - zooms out, and 0 or Home resets it. Losing focus, resizing or leaving the warehouse cancels a pending drag.

Sơn stands near Returnables, Minh near GR, Trí near Pallet Interlocking and Bách near PutAway. Ngân patrols a route around the warehouse. The five named people have clickable geometry and separate name buttons, so they remain discoverable at smaller map scales. Labels are laid out in screen space and adjusted around occupied regions. Anonymous operators support aisle and work activity without the named greeting interaction.

PHRASES in js/twin.js supplies both clicked greetings and spontaneous speech. The current implementation expects at least two phrases to choose a different random message. Greeting timers and random-chat scheduling are nearby. User-triggered greetings take priority over ambient chatter. Reduced motion disables spontaneous chatter, and hidden or inactive warehouse views pause it.

10. Operator animation

js/operators.js generates articulated body parts and changes their joint poses for walking, turning, work and greetings. Walking is based on distance travelled rather than simply oscillating limbs against elapsed time. This helps align planted feet with movement and reduces sliding. Stationary poses include small shifts and gestures; wrapping and scanning have separate work poses.

Movement and pose state come from the activity model. Operators can pause for hover, keyboard focus or a greeting. Rig updates are limited to roughly 30 updates per second while scene translation can continue on the animation frame schedule. This is an update strategy, not a guarantee of a particular displayed frame rate on every device.

11. Forklifts, pallets and pedestrian yielding

The activity model in js/logistics.js is separate from SVG rendering. Two forklifts follow task programs built from move, turn, lift, wait, pickup, drop and pause operations. Each forklift keeps its current step, position, angle, lift height, speed and carried cargo. Lifting and ownership transfer occur while the forklift is stopped.

Each pallet has one owner at a time. Owners include truck-in, a forklift ID, blue-buffer, rack-ground, rack-upper, green-buffer and truck-out. This invariant prevents a pallet from appearing in two locations at once or remaining behind after pickup. Wait conditions coordinate free storage positions, truck readiness and clear approaches.

The receiving program unloads a truck into the blue receiving buffer and transfers the pallet to a lower rack position. The other program moves cargo through upper storage, retrieves it, transfers it to the green buffer and then loads an outbound truck. Model time advances incrementally, with controlled movement, turning and forklift rates rather than teleporting between task endpoints.

Pedestrian yielding uses geometric proximity and directional checks. A forklift pauses when an operator is in its path; that operator selects an aside position and moves away, after which the vehicle resumes. These are scripted simulation rules, not a safety-certified vehicle controller or a general physics/pathfinding engine.

12. Trucks, doors and receiving gate

js/vehicles.js describes approach, reverse docking, loading and departure poses. The trucks turn into the approach, reverse rear-first to the dock and depart cab-first. Trailer meshes include a floor, walls, roof and hinged rear doors, rather than a single solid box. Door faces are rotated and depth-sorted with the rest of the vehicle.

The receiving and shipping clocks use nominal 24-second and 48-second CSS cycles. holdDock() can pause loading while a forklift handles cargo. clearLane() coordinates the next truck with an occupied approach. Consequently, an actual delivery can take longer than the nominal cycle. A loaded pallet attached to truck-out follows the outbound truck's pose until it leaves.

The inbound blue container's receiving shutter opens for the dock/unloading sequence and forklift clearance, then closes as appropriate when the truck leaves. The outbound container remains open. These are coordinated visual/model events; they are not connected to physical gate sensors.

13. Pallet wrapping

The main yellow wrapper contains a wooden pallet, stacked cartons, a rotating turntable and a film roll. The load rotates around the turntable center in plan coordinates. Film faces follow the rotated carton outline, while the feed strip joins the roll to a visible tangent corner of the load. Roll height and film height rise together to keep the feed aligned.

The current ambient cycle lasts about 14 seconds: approximately 10 seconds of eased rotation, totaling three turns, followed by a finished pause. The band builds upward and avoids passing through the cartons. The attendant changes work pose during wrapping. The guided handoff has its own shorter transfer/wrap choreography and does not move the ambient scene's cargo records.

14. Responsive layout, themes and logos

The home view fits the viewport beneath the header. CSS uses Grid, Flexbox, clamp(), container-size units and container/media queries. Main.js measures the real header height into --topbar-h through ResizeObserver, avoiding an incorrect fixed offset after wrapping, translation or browser zoom.

The lower-left caption remains anchored to the home canvas. fitHomeCanvas() measures its actual translated size and scales it when necessary. Narrow screens reserve a separate region for the map. Very short landscape windows retain the main heading and start action while hiding secondary caption elements. On narrow portrait screens, map controls sit above the plan so they do not cover station shortcuts. Station terminals and Project Detail can scroll internally when their content exceeds the available height.

Theme values live in css/tokens.css and are selected with html[data-theme]. The header toggle changes the theme and its accessible label. The initial theme follows prefers-color-scheme. English/Vietnamese text comes from js/i18n.js through data-i18n and data-i18n-aria attributes. Both dictionaries must be updated for ordinary interface copy.

The Schneider Electric and RMIT School of Science, Engineering & Technology logos are local assets under assets/. Both supplied PNGs already contain alpha transparency; their header containers therefore use transparent backgrounds. The RMIT logo is embedded in an inline SVG. A clipped, white-ink copy overlays only its navy wordmark in dark mode, preserving the red emblem and school panel. Schneider's green is brightened slightly in dark mode. The original image files are not altered. Placement and theme treatment are controlled in css/layout.css, and the portable builder embeds both PNGs as data URLs. If replacing the RMIT artwork with a different composition, update the wordmark clip rectangle in index.html.

15. Accessibility

The app provides a skip link, semantic buttons, keyboard focus styling, translated accessible labels, active-navigation aria-current values and live status/progress text. Name callouts provide larger operator targets than the small SVG figures alone. Map controls have keyboard alternatives. Dialogs manage focus, use inert for the background and provide Escape/skip actions.

prefers-reduced-motion reduces or bypasses movement, and visibilitychange pauses work when the page is hidden. Colour distinguishes station identity and status, while text, icons and controls provide additional cues. These features are implemented and exercised by targeted checks; they do not constitute a complete accessibility

certification or a screen-reader audit across all browser combinations.

16. Performance techniques and limits

Static SVG geometry is generated during scene construction. Dynamic rendering reuses mesh nodes and changes attributes rather than rebuilding the entire warehouse every frame. Vehicle meshes cache rounded pose signatures to avoid repeated work for unchanged poses. Numeric projection avoids repeatedly parsing coordinate strings. Operator poses update at a lower frequency, and label collision layout runs around 10 times per second.

Animations pause outside the active scene and when the document is hidden. The activity loop uses elapsed time and bounds large frame deltas. Controlled requestAnimationFrame clocks in tests make long sequences reproducible. Performance depends on viewport size, SVG workload, browser and hardware; the project does not promise a universal 60 fps result. Re-profile representative physical devices before an exhibition or production rollout.

17. Data handling and production boundaries

Runtime application scripts do not send demo records through fetch, XMLHttpRequest or WebSocket, and do not store visitor data persistently. Locally bundled assets avoid a runtime CDN dependency. Escape helpers are used when composing dynamic HTML from strings; fixed UI templates also use innerHTML. If external data is introduced, audit every insertion point rather than assuming the current sample-data design safely accepts arbitrary content.

This repository has no authentication, authorization, server-side validation, audit log, transaction database or production inventory reconciliation. Deploying a real warehouse system would require those services, a defined data model and API contract, barcode integration, error/retry handling, security review and operational acceptance testing. That future work is separate from the current demonstration.

18. Running and building

Open index.html directly in a current browser to use the source version. Open preview-single-file.html to use the portable version. The preview includes the application's CSS, JavaScript, favicon and header logos; separate legal pages remain separate files if their links are used.

For development and browser checks, install the locked Node dependencies:

```
npm ci
npm run build:preview
```

The Node builder reads index.html, inlines local styles and scripts, embeds logo PNGs and the favicon, and writes preview-single-file.html. Run it after changing source files or assets. Do not edit the generated preview by hand. build-preview.py provides the equivalent preview workflow for a Python environment.

Static HTTP hosting can serve the project without an application server. Fragment routes do not need special rewrite rules. Browser checks default to the local Windows Edge executable; set BROWSER_PATH to the installed compatible browser when running elsewhere. The source app itself does not require Node or Python at runtime.

To update this guide, edit docs/technical-guide.md, run npm run build:docs to regenerate Word/HTML, and run npm run build:docs:pdf to export the PDF with Edge or the browser selected by BROWSER_PATH.

19. Verification commands

Command	Scope
npm run check:navigation	Back/Forward, direct fragments, reload, animation cancellation, page return, header layout and logo loading for source/preview over file and HTTP
npm run check:experience	Responsive views, project challenge and the full GR → PutAway → Interlocking → home journey
npm run check:responsive	Earlier broad responsive and workflow checks
npm run check:interactions	Click, keyboard, touch dragging and zoom interactions
npm run check:crew	Operator placement and truck manoeuvres
npm run check:operators	Gait, work/greeting poses and reduced motion
npm run check:chatter	Ambient chat, click priority and layout
npm run check:wrapping	Film alignment, clearance, wrap cycle and station targeting
npm run check:grounding	Floor contact, rack supports, wrapper drum and paint order
npm run check:dock	Receiving dock and gate cycle checks
npm run check:logistics	Extended model simulation and cargo ownership
npm run check:logistics-browser	Rendered logistics, routes and wrapping
npm run check:lane-priority	Forklift/container access and truck hold/release coordination

The navigation suite checks 14 viewport sizes, two languages and two themes for each of four source/preview and file/HTTP combinations: 224 header layouts. The experience suite checks 22 sizes × two languages × two themes × five views × two app files: 880 view layouts. Screenshots and JSON reports are written below artifacts/.

These are representative automated Chromium checks, not proof of behaviour on every physical device. Test current Safari/iOS and Firefox separately if they are part of the required support matrix. Some older scripts preserve historical expectations; use the targeted suite relevant to a change and investigate failures against current intended behaviour.

20. Common maintenance changes

Change	Edit
--------	------

Operator speech	PHRASES in js/twin.js; retain at least two entries with the current random-selection code
English/Vietnamese UI text	Corresponding keys in both dictionaries in js/i18n.js
Top-bar logos	Asset references in index.html and logo/theme rules in css/layout.css
Warehouse object location	Geometry in js/twin.js and related details in js/warehouse-environment.js; verify depth and route clearance
Operator route or home position	TEAM and people configuration in js/twin.js, with movement handling in js/logistics.js
Forklift task order	RECEIVING / SHIPPING programs in js/logistics.js
Vehicle appearance or trailer doors	Mesh definitions in js/twin.js / js/logistics-renderer.js and drawing in js/vehicles.js
Wrapping choreography	js/logistics-renderer.js; keep roll, film, load and floor heights consistent
Detailed navigation forklift	js/transition-forklift.js and transition CSS
Guided demo handoffs	js/handoff.js, css/handoff.css and orchestration in js/main.js
Sample docket, pallet or packing data	Constants and checks in js/demos.js or js/challenge.js
Theme colours or global spacing	css/tokens.css
Browser navigation	js/router.js; rerun check:navigation and check:experience

After a change, run the relevant checks, regenerate the preview and open both versions. Inspect light/dark mode, Vietnamese/English, a phone-sized viewport and a short landscape viewport. For scene changes, inspect occlusion and floor contact as well as movement. For route changes, interrupt the transition with browser Back and confirm no overlay or stale timer blocks the app.

21. Troubleshooting

Symptom	What to inspect
Preview shows old content	Rebuild preview-single-file.html, then reload the correct file
Back opens the wrong view or a curtain stays visible	Fragment URL, Router callbacks, pending timers and handoff cleanup
Warehouse stops after returning	Active-view subscription, visibility state, reduced-motion preference and pageshow handling
Rack or wrapper appears to float	Floor z, support height, contact shadows and SVG paint order
Forklift waits for a long time	Current program step, named wait condition, storage owner and truck dock/clear state

Pallet appears twice	Ownership transfer and model-to-renderer mapping
Logo looks unreadable	Transparent image alpha, preserved school-panel lettering, theme rules and intrinsic image proportions
Header covers the scene	Measured --topbar-h and responsive top-bar grid
Operator appears behind equipment	Check intentional layer occlusion before changing its route
Tests cannot launch a browser	Installed Edge/Chromium path, BROWSER_PATH and local execution permissions

This guide documents the implemented demo rather than a proposed production system. Use the code and the latest targeted test report as the final reference when behaviour changes.